

Gratitude

Server host procedure

Linux VM deploy checklist

September 16, 2026

This document covers only how to host **Gratitude** on a Linux server. It assumes you have SSH access (IP address, OS, username and password or key).

Contents

1	What you are hosting	2
2	Access the server	2
3	Install base software	2
4	Deploy the application	3
4.1	Clone and install	3
4.2	API environment	3
4.3	Database migrate and seed	3
4.4	Build the web UI	3
4.5	Keep uploads on disk	4
5	Run the API as a service	4
6	nginx reverse proxy	4
7	Same host for API and web (optional)	5
8	Updates after go-live	6
9	Smoke checks	6
10	Common host failures	6

1 What you are hosting

Gratitude needs two parts on the server:

1. **API** - Node.js app in `api/` (port 3001 by default). Serves JSON, auth and uploaded card images under `/uploads`.
2. **Web** - Built React app from `web/`. Served as static files (nginx recommended).

Also required:

- PostgreSQL database
- Durable disk for `api/uploads/` (do not rely on ephemeral cloud disks)

One VM can also host other projects. Keep separate folders, databases and upload directories per app. Use nginx to route by hostname.

2 Access the server

1. Confirm you have: public IP, SSH username, password or SSH key and OS (Ubuntu preferred).
2. From your PC:

```
ssh USERNAME@SERVER_IP
```

3. Change the shared password on first login if it was emailed to you.
4. Prefer SSH keys for ongoing access. WinSCP is optional; SSH is enough.

3 Install base software

On Ubuntu/Debian-style hosts:

```
sudo apt update
sudo apt install -y git curl build-essential nginx
```

Install Node.js 20+:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

Install PostgreSQL (on this VM or use a managed database):

```
sudo apt install -y postgresql postgresql-contrib
sudo -u postgres createuser grat WITH PASSWORD 'CHOOSE_A_STRONG_PASSWORD';
sudo -u postgres createdb project_grat -O grat
```

Open firewall ports if using ufw:

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

4 Deploy the application

4.1 Clone and install

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/SeniruR/project_grat.git
sudo chown -R $USER:$USER /var/www/project_grat
cd /var/www/project_grat

npm install
cd api && npm install
cd ../web && npm install
```

4.2 API environment

Create `/var/www/project_grat/api/.env`:

```
DATABASE_URL="postgresql://grat:PASSWORD@localhost:5432/project_grat?schema=public"
PORT=3001
HOST=0.0.0.0
JWT_SECRET="REPLACE_WITH_LONG_RANDOM_SECRET"
CORS_ORIGIN="https://YOUR_WEB_HOSTNAME"
PUBLIC_API_URL="https://YOUR_API_HOSTNAME"
AUTH_MODE=dev
DIRECTORY_MODE=mock
MAIL_MODE=mock
```

Notes:

- `PUBLIC_API_URL` must be the **API** public URL (or the same host if nginx proxies both). Never point it only at a static frontend that cannot serve `/uploads`.
- `CORS_ORIGIN` must include the browser origin of the web UI.
- For real email later set `MAIL_MODE=smtp` or `graph` and the related secrets.

4.3 Database migrate and seed

```
cd /var/www/project_grat/api
npx prisma generate
npx prisma migrate deploy
npm run db:seed
```

4.4 Build the web UI

Set the API URL for the frontend build then build:

```
cd /var/www/project_grat/web
export VITE_API_URL="https://YOUR_API_HOSTNAME"
npm run build
```

Output is in `web/dist/`.

4.5 Keep uploads on disk

```
mkdir -p /var/www/project_grat/api/uploads
```

Back up this folder. If the VM is recreated without this data, card images break while database rows remain.

5 Run the API as a service

Install a process manager (example with systemd). Create `/etc/systemd/system/gratitude-api.service`:

```
[Unit]
Description=Gratitude API
After=network.target postgresql.service

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/project_grat/api
EnvironmentFile=/var/www/project_grat/api/.env
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Adjust `User` to the account that owns the files. Then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now gratitude-api
sudo systemctl status gratitude-api
curl http://127.0.0.1:3001/health
```

6 nginx reverse proxy

Example: web on `grat.example.com`, API on `api-grat.example.com`.

Web site:

```
server {
    listen 80;
    server_name grat.example.com;
    root /var/www/project_grat/web/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

API site:

```
server {
    listen 80;
    server_name api-grat.example.com;
```

```
client_max_body_size 20M;

location / {
    proxy_pass http://127.0.0.1:3001;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

Enable and reload:

```
sudo ln -s /etc/nginx/sites-available/gratitude /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Add TLS with Certbot when DNS points at the server:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d grat.example.com -d api-grat.example.com
```

Then set `CORS_ORIGIN`, `PUBLIC_API_URL` and `VITE_API_URL` to the `https://` URLs and rebuild the web app if needed.

7 Same host for API and web (optional)

If you use one hostname only:

```
server {
    listen 80;
    server_name grat.example.com;
    root /var/www/project_grat/web/dist;
    index index.html;
    client_max_body_size 20M;

    location /uploads/ {
        proxy_pass http://127.0.0.1:3001/uploads/;
        proxy_set_header Host $host;
    }

    location /api/ {
        # only if you later mount the API under /api
    }

    location / {
        # Prefer separate API host. If API is same origin via path,
        # proxy API routes explicitly instead.
        try_files $uri $uri/ /index.html;
    }
}
```

Simplest production pattern remains: **separate hostnames** for web and API as in the previous section. Then:

```
PUBLIC_API_URL=https://api-grat.example.com
CORS_ORIGIN=https://grat.example.com
VITE_API_URL=https://api-grat.example.com
```

8 Updates after go-live

```
cd /var/www/project_grat
git pull
cd api && npm install && npx prisma migrate deploy
cd ../web && npm install
export VITE_API_URL="https://YOUR_API_HOSTNAME"
npm run build
sudo systemctl restart gratitude-api
```

Do not delete api/uploads/ during updates.

9 Smoke checks

1. `curl https://YOUR_API_HOSTNAME/health` returns OK.
2. Open the web URL in a browser and sign in.
3. Upload or import a card as a designer and confirm images appear after refresh.
4. Restart the API service and confirm images still load (proves disk persistence).
5. Send a test card (mock or SMTP as configured).

10 Common host failures

Symptom	Fix
Cannot SSH	Check IP, username, port 22 and firewall.
API migrate fails	Fix DATABASE_URL; ensure Postgres is running and reachable.
Broken card images	Uploads folder missing or wiped; restore backup or re-upload. Check PUBLIC_API_URL.
Browser CORS errors	Add the web origin to CORS_ORIGIN and restart API.
Web loads but API 404	Wrong VITE_API_URL at build time; rebuild web.
